

10. «Эпизодическое» объектно-ориентированное программирование

ООП – Организация Освобождения Палестины. **Аббревиатура.**

Резонный вопрос – почему так поздно приступаем к знакомству с ООП? Я тоже считаю, что некоторые главы книги только бы выиграли от их изложения в объектно-ориентированной нотации. Но, сказавши «а», следовало бы сказать и «б», т.е. пришлось бы полностью изложить принципы ООП, а это было бы не совсем правильно:

- эта технология ориентирована на создание уже достаточно больших проектов, хотя отдельные части проекта (например, методы классов) все равно разрабатываются в рамках традиционной технологии структурного программирования. Поэтому, на мой консервативный взгляд, чтобы почувствовать преимущества технологии ООП, надо иметь опыт разработки проектов определенной сложности в традиционной технологии;
- многие механизмы объектно-ориентированного Си прекрасно иллюстрируются средствами классического Си, чтобы понимать первое, нужно знать второе;
- все предыдущие главы иллюстрированы небольшими по объему программами, для которых объектно-ориентированная нотация (именно как для примеров) не обязательна;
- ООП – это постановка процесса программирования «с ног на голову», (или с головы на ноги), а это лучше сделать не в середине изложения материала;

И, наконец, такой «монстр» как Си++, пытающийся сочетать в себе все и вся, имеет не совсем удобную, излишне открытую и довольно громоздкую объектно-ориентированную нотацию. Поэтому данный материал следует рассматривать как приглашение к знакомству с тотальными средами ООП, например, Java или C#.

10.1 Объекты и классы

Объект, метод, класс: определения и свойства

«Классами называются большие группы людей, различающиеся по их месту в исторически определенной системе общественного производства, по их отношению) к средствам производства, по их роли в общественной организации труда, а следовательно, по способам получения и размерам той доли общественного богатства, которой они располагают» Ленинское определение классов.

В 5.3 мы уже сделали попытку привязать к структурированному типу терминологию «класс – свойство – метод - объект». Теперь настало время окончательно расставить «все точки над ё». Для начала рассмотрим несколько определений класса.

Технологическое определение класса. Технология ООП, прежде всего, накладывает ограничения на способы представления данных в программе и их взаимодействие с алгоритмической компонентой (функциями). Любая программа отображает в своих данных состояние **внешних объектов программирования**. Это могут быть как физические объекты внешней среды, так и логические программные сущности (например, файлы). Для этого можно использовать различные способы, можно «размазать» свойства объекта по различным структурам данных. Можно исходить из того, что каждому объекту будет соответствовать своя собственная структура данных, в которой содержатся все элементы описания свойств внешнего объекта

программирования. Такую структуру данных можно аналогично назвать **объектом**. Функции, работающие с объектом и получающие в качестве обязательного параметра указатель на структуру данных, называются **методами**. Совокупность описания объектов одного типа и методов работы с ними называется **классом**.

Объект – структура данных, содержащая описание свойств внешнего объекта программирования. **Метод** – функция, работающая с объектом. **Класс** – описание структуры объекта и методов работы с ним.

Строго говоря, реализовать идеи ООП можно в классической среде программирования, соблюдая дух, а не букву технологии. Например, **библиотека функций**, работающая на общую структуру данных, может в первом приближении считаться классом.

Синтаксическое определение объекта и класса - структура со встроенными функциями. Структурированная переменная со встроенными в нее функциями (см. 5.3) идеально подходит под определение объекта. Не нужно пытаться изобретать велосипед, достаточно назвать вещи своими именами. Структурированная переменная – это объект, функции, встроенные в структурированный тип – это методы класса, а само определение структурированного типа (включающее и определение встроенных функций) – есть класс.

Синтаксическое определение класса – тип данных, определяемый программистом. Класс представляет собой описание структуры объектов одного вида с набором методов их обработки. Аналогия с типом данных здесь напрашивается сама собой. Тип данных – это форма представления данных с набором операций. Отличие состоит в том, что тип данных либо уже определен в языке, либо формально составляется из уже существующих (но без внутреннего программирования). Значит, класс можно определить как тип данных, определяемый программистом. Тогда объект – это переменная класса. Эта трактовка закреплена в языке: синтаксис определения переменных и объектов почти полностью идентичен.

Класс – тип данных, определяемый программистом, **объект** – переменная класса

Прописные истины объектно-ориентированного подхода

Объектно-ориентированный подход не ограничен синтаксисом. Следует соблюдать не только букву, но и дух ООП. Но даже в самой реализации понятий класса и объекта в языке программирования имеется много очевидных, но не всегда упоминаемых вещей, которые следует помнить. Попробуем их здесь перечислить.

Контекст класса (текущего объекта). Понятие контекста мы уже вводили применительно к функции (1.6). Контекст (окружение) – это набор имен (объектов языка), которые можно использовать непосредственно, без указания «пути доступа». Контекстом функции являются ее формальные параметры и локальные переменные. Каждый класс определяет свой контекст – это имена элементов данных и методов класса. Этот контекст возникает, поскольку программные компоненты класса имеют дело с текущим объектом, умолчание касается именно его. Любой элемент контекста может адресоваться как просто по имени, так и через указатель **this**, например **n** и **this->n**.

Классы и порождение объектов. Класс – это описание объектов, объект – это инсталляция (отображение) класса в памяти. Можно, конечно, это понимать буквально,

как создание экземпляра класса со всеми его «потрохами» - данными и методами. В реальности же проекция класса на традиционную компьютерную архитектуру выглядит таким образом:

для каждого объекта создается экземпляр данных;

методы класса, с которыми работает объект, представляют собой единственный экземпляр программного кода в сегменте команд, который одинаково выполняется для всех объектов (разделяется ими);

при вызове метода объект, для которого он выполняется, идентифицируется указателем текущего объекта **this**, задающим контекст текущего объекта.

Таким образом, связка «объект-метод» преобразуется в традиционную последовательность действий: «вызов функции – метода класса с фактическим параметром – указателем на текущий объект».

class A {	// Эквивалентно:
int a;	// struct A { int a; };
public: void F(){ a++; }	// void A::F(A *this) { this->a++; }
};	
	// A DD;
A DD; DD.F();	// A::F(&DD); this=ⅅ

Класс – граница ответственности транслятора и программы. При наличии разнообразия определений класса без ответа остался основной вопрос: где проходит граница ответственности между транслятором и программой (следовательно, и программистом, ее пишущим). Эта граница между технологическим и синтаксическим определением класса, т.е. между структурированной переменной – синтаксическим объектом и связанной с ней структурой данных – объектом технологическим. Транслятор несет ответственность за синтаксический объект, обеспечивая, прежде всего, действия, связанные с его копированием: **присваивание, передачу по значению в функцию и возвращение его в виде значения-результата**. За все остальное несет ответственность программист, точнее написанный им программный код класса:

- если элементы данных класса имеют взаимосвязанные значения, то класс должен поддерживать установленные для них соглашения;
- если объект данных класса ссылается на внешние структуры данных, то при синтаксическом копировании объекта необходимо обеспечить независимость связанной структуры данных в объекте-копии (создать ее копию или обеспечить разделение – см. «конструктор копирования»);
- если объект содержит идентификаторы каких-либо внешних ресурсов (например, номер коммуникационного порта), то действия класса должны быть аналогичными.

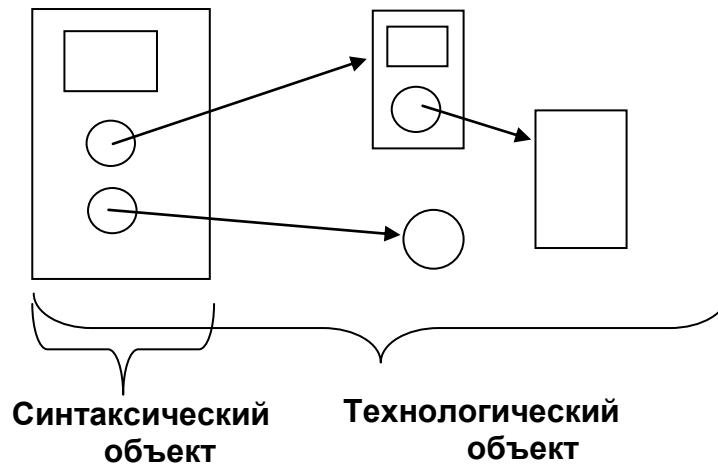


рис. 101-1. Объект: граница ответственности транслятора и программы

Модульное проектирование и ООП. Объектно-ориентированное программирование по своей природе является модульным. На поверхности лежит очевидный факт: наличие большого количества примитивных методов работы с объектом позволяет, комбинируя их, получать новые при минимуме затрат. Но есть еще другая возможность: при разработке метода можно создавать локальные объекты того же самого класса, вызывая для них уже известные методы, что также обеспечивает повторное использование уже написанного кода.

Свойства объекта: закрытость, независимость, универсальность, целостность. Следующие принципы, хотя и поддерживаются синтаксически, относятся все-таки к элементам технологии, ибо программист может их соблюдать или игнорировать по своему усмотрению. Следовательно, они относятся к духу, а не к букве ООП.

Закрытость. Данные класса должны быть максимально закрыты, внешний пользователь не должен подозревать, что находится внутри объекта, и, тем более, на него не должны переноситься проблемы, связанные с их размещением, корректностью и т.д.. Нарушением принципа закрытости является использование в качестве входных параметров методов структур данных, аналогичных внутреннему представлению, лучше использовать объекты того же класса.

```
class poly{
    int n;
    double *pd;                // Внутренняя СД – дин. массив коэффициентов
public: void add(double D2[], int n2){} // Нарушение закрытости – параметр – внутренняя СД
    void add(poly &T){}         // Правильно: параметр – объект того же класса
};
```

Независимость. Внутренние данные объекта должны быть «личной собственностью» объекта. При выполнении операций, связанных с копированием его содержимого (присваивание, передача по значению) класс должен обеспечивать создание независимых копии связанных с объектом данных. Возможен и другой вариант: несколько объектов могут разделять общую структуру данных. Но тогда в классе должен быть прописан механизм такого разделения и синхронизации доступа.

Универсальность. По отношению к данным универсальность понимается как отсутствие ограничений на их размерности и, как следствие, использование динамических структур данных во внутреннем представлении объектов. Класс при этом

должен решать проблемы, связанные с динамическим распределением памяти, ее занятием, освобождением и перераспределением.

По отношению к методам это означает, что интерфейс класса (набор методов) должен быть максимально разнообразен, методы должны сочетаться в любых комбинациях, давая широкое разнообразие возможностей работы с объектом.

Целостность. Содержимое объекта должно быть всегда корректно с точки зрения соглашений, установленных классом для возможных вариантов его внутреннего представления, за этим прежде всего должны следить конструкторы и деструктор, другие методы тоже не должны «оставлять после себя» неправильных объектов.

Полезный совет: желательно избегать многообразия форм представления внутренних данных объекта. Чем их меньше, тем проще обеспечить его целостность и корректность. Например, лучше иметь фиктивный динамический массив, чем **NULL**-указатель. В примере с классом степенного полинома «пустой» полином лучше представить динамическим массивом с единственным нулевым коэффициентом.

```
class poly{
    int n;
    double *pd;
public: poly(){ n=0; pd=NULL; } // Внутренняя СД – дин. массив коэффициентов
    poly(){ n=0;                // Нежелательно: NULL – отсутствие массива
        pd=new double[1];      // Правильно: полином с a(0)=0
        pd[0]=0;
    };
};
```



«Ложась спать, программист ставит рядом два стакана: один полный – если захочет пить, и один пустой – если не захочет». **Анекдот в тему.**

Объект – замкнутые, логически непротиворечивые, всегда корректные данные с четко определенным универсальным интерфейсом доступа к ним

Настало время посмотреть, как перечисленные требования учесть на практике. В качестве примера рассмотрим класс степенного полинома. Он имеет нетривиальное предметное наполнение, а с другой стороны, достаточно прост в реализации. Полином **n**-ой степени представляет собой выражение вида $a_0 + a_1x + a_2x^2 + \dots + a_nx^n$ или $\sum a_i x^i$. Для его представления и манипуляций с ним в объекте достаточно хранить массив его коэффициентов. Требование **универсальности** сразу же предполагает произвольную размерность и динамический массив, требование **независимости** – каждый объект обладает собственным динамическим массивом (разделение не допускается), требование **закрытости** – этот массив не доступен извне. Если в операции участвуют более одного полинома, то все они передаются как объекты, а не как динамические массивы. И наконец, в классе есть **текущий объект**, над которым полагается выполнение методов, он не может быть «свадебным генералом».

```
//-----101-01.cpp
// Класс степенного полинома – заголовок класса (объявление)
struct poly{
    int n; // степень полинома
    double *pd; // динамический массив коэффициентов
    ...
    double &get(int k); // получение ссылки на коэффициент
    void add(poly &T); // сложение объектов (1=1+2)
    void mul(poly &T); // умножение объектов (1=1+2)
    ...};
```

Целостность объекта. Конструктор. Деструктор

Требование целостности и корректности объекта означают, что объект – это нечто большее, чем просто переменная. При создании переменной ее инициализация вовсе не обязательна, в то время как создание объекта должно сопровождаться установлением его начального состояния (инициализация данных, резервирование памяти, ресурсов, установление связей и т.д.). Аналогичные обратные действия необходимо выполнить при его уничтожении перед освобождением памяти. С этой целью в классе вводятся специальные методы – **конструкторы** и **деструктор**. Их имена совпадают с именем класса. Конструкторов для данного класса может быть сколь угодно много, они отличаются формальными параметрами, деструктор же всегда один и имеет имя, предваренное символом "~". Если конструктор имеет формальные параметры, то в определении переменной-объекта после ее имени должны присутствовать в скобках значения фактических параметров.

```
//-----101-01.cpp
// Класс степенного полинома – конструкторы и деструктор
struct poly{
int n;                // степень полинома
double *pd;           // динамический массив коэффициентов
poly(){               // "пустой" полином - нулевой степени
    n=0;              // с нулевым коэффициентом
    pd=new double[1];
    pd[0]=0;}
poly(int m){           // полином заданной степени
    n=m;              // с нулевыми коэффициентами
    pd=new double[n+1];
    for(int i=0;i<=n;i++)
        pd[i]=0; }
poly(int n0,double p[]){ // конструктор из массива коэффициентов
    load(n0,p); }      // используется вспомогательный метод load
poly(poly &T){         // конструктор "объект из объекта"
    load(T.n, T.pd); } // (конструктор копирования)
~poly(){               // деструктор
    delete []pd; }
```

Момент вызова конструктора и деструктора определяется временем создания и уничтожения объектов:

- для статических и внешних объектов - конструктор вызывается перед входом в **main**, деструктор - после выхода из **main()**. Конструкторы вызываются в порядке определения объектов, деструкторы - в обратном порядке;
- для автоматических объектов - конструктор вызывается при входе в функцию (блок), деструктор - при выходе из него;
- для динамических объектов - конструктор вызывается при выполнении оператора **new**, деструктор - при выполнении оператора **delete**.

В Си++ возможно определение массива объектов класса. При этом конструктор и деструктор автоматически вызываются в цикле для каждого элемента массива и не должны иметь параметров. При выполнении оператора **delete** для указателя на массив объектов его необходимо предварять скобками.

```
struct poly { ..... }; // определение класса
double D[]={1,2,3,4};
poly a,b(6),c(3,D);     // Статические объекты – конструкторы
                        // пустой полином, заданной размерности и из массива
poly *p,*q;             // Указатели на объект
void main(){
```

```

poly    c,d(c);           // Автоматические объекты
p = new poly;             // Динамический объект
q = new poly[n];          // Динамический массив объектов
delete p;                 // Уничтожение динамического объекта
delete []q;               // Уничтожение динамического массива объектов
}                          // Уничтожение автоматических объектов

```

Замечание: процесс конструирования «вложен» в процесс выделения памяти под переменную. Конструктор вызывается сразу же после выделения памяти, а деструктор – перед ее освобождением.

Безымянные объекты. Программе зачастую требуются «одноразовые» объекты, для которых только и требуется, что передать их на вход функции или метода, либо использовать их в выражении в качестве аналога константы. Такой объект можно создать, указав имя класса и в скобках фактические параметры конструктора.

```

//----- Безымянные объекты
struct A{
    int a;
    A(int a1){a=a1;}           // Конструктор
    A &INC(){ a++; return *this; } // Метод класса - инкремент
    void show(){ printf("%d\n",a); }
};
void F(A bb){ bb.a++; bb.show(); }
void main(){
    F(A(5));                  // Фактический параметр – безымянный объект
    A(2).INC().INC().show();  // Вызов методов для безымянного объекта
}

```

Класс – структурированный тип с ограниченным доступом

«Настоящий» классы в Си++ отличается от структурированного типа одной единственной мелочью: в классе вводятся ограничения доступа. Естественно, это синтаксические ограничения, и при желании их можно исключить простым редактированием заголовка класса. Это «дисциплинирующие» ограничения, позволяющие установить зоны ответственности программистов – разработчика класса и пользователя класса, обеспечить необходимую **закрытость**.

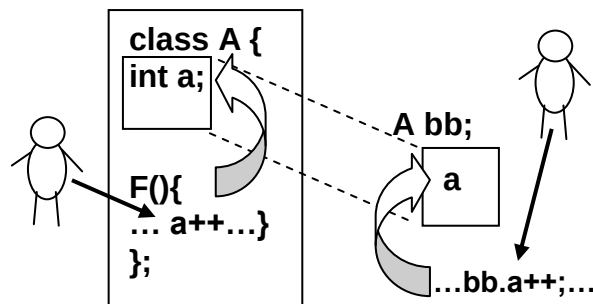


рис. 101-2. Внешний и внутренний доступ к объекту

В процессе программирования класса участвуют два действующих лица с различной компетенцией: разработчик класса, пишущий его внутренний код, и пользователь класса – программист, создающий объекты этого класса и вызывающий для них его методы. Но ограничения касаются не самих программистов, а кода, который они создают. **Внутреннее программирование** класса – это разработка программного кода, который находится в теле разрабатываемого класса (точнее, в теле его методов). **Внешнее программирование** – это разработка кода вне тела проектируемого класса, который создает объекты класса, работает с данными этих объектов и вызывает методы.

Формально класс отличается от структурированного типа ключевым словом **class** (вместо **struct**) и наличием двух областей доступа в теле класса:

- закрытая (личная) часть, допускает только внутреннее программирование и закрыта при доступе через объект вне класса. По правилам синтаксиса закрытая часть начинается сразу же вслед за заголовком класса. Она также может быть обозначена меткой **private**;
- открытая (общая) часть класса допускает любой доступ, в том числе и внешний. Она всегда явно обозначается меткой **public**.

Стандартным является размещение данных объекта в личной части, а методов - в общей части класса. Тогда закрытая личная часть определяет содержимое объекта, а методы общей части образуют интерфейс объекта «к внешнему миру».

```
//-----101-01.cpp
// Класс степенного полинома
class poly{                // закрытая часть (по умолчанию) – данные
int n;                    // степень полинома
double *pd;               // динамический массив коэффициентов
public:...                // метка открытой части
```

Другие варианты размещения данных и методов в личной и общей части класса встречаются реже, но тоже обоснованы:

- в общей части класса могут быть размещены данные, изменение которых пользователем класса не может привести к катастрофическим последствиям (например, цвет фигуры). Естественно, что класс будет учитывать изменение этих данных только при вызове методов (например, при перерисовке фигуры);
- в личной части класса может быть размещен внутренний метод, необходимый для работы самого класса. Это могут быть вспомогательные действия, вынесенные за пределы конкретных методов, либо такие операции, корректное выполнение которых требует дополнительных действий.

Таким образом, в первом приближении класс отличается от структуры четко определенным интерфейсом доступа к его элементам. И наоборот, структура - это класс без личной части.

Иногда требуется ввести исключения из правил доступа, когда некоторой функции или классу требуется разрешить доступ к личной части объекта класса. Тогда в определении класса, к объектам которого разрешается такой доступ, должно быть объявление функции или другого класса как «дружественных». Это согласуется с тем принципом, что сам класс определяет права доступа к своим объектам «со стороны».

Объявление дружественной функции представляет собой прототип функции, переопределяемой операции или имя класса, которым разрешается доступ, предваренное ключевым словом **friend**.

```
// Классы и функции, дружественные классу А
class A {
    int x;    // Личная часть класса
    ...      // Все «друзья» имеют доступ к x
    friend class B;
    friend void C::fun(A&);
    friend void xxx(A&,int);
    friend void C::operator+(A&);
};
```



«Друг – это тот, кто имеет исключительное право лезть тебе в душу (личную часть) в любое время».

Возвращаясь к классу полиномов, сразу же заметим, что в нем можно по большей части обойтись без дружелюбности. Закрытость же касается только данных (размерность и указатель на динамический массив), а также методов, связанных с управлением динамической памятью при изменении размерности полинома.

Задача управления динамической памятью должна быть решена раз и навсегда в начале проектирования класса, чтобы в дальнейшем к ней не возвращаться. Удобнее всего сделать это в виде внутренних методов управления размерностью объекта. Предпочтительнее также создавать дополнительные локальные объекты требуемой размерности, нежели создавать в явном виде динамические структуры данных.

```
//-----101-01.cpp
// Класс степенного полинома
class poly{                                // закрытая часть (по умолчанию) – данные
    int n;                                // степень полинома
    double *pd;                            // динамический массив коэффициентов
void load(int n0, double p[]){
    n=n0;                                // закрытый метод загрузки массива
    pd=new double[n+1];                  // - не всегда корректно вызывается
    for(int i=0;i<=n;i++)
        pd[i]=p[i]; }
void extend(int n1){                      // увеличение размерности полинома
    if (n1<=n) return;
    double *pd1=new double[n1+1];
    for (int i=0;i<=n;i++) pd1[i]=pd[i];
    for (;i<=n1;i++) pd1[i]=0; // прописать старшие коэффициенты нулями
    n=n1;
    delete []pd;                          // удалить старый массив
    pd=pd1;                                // считать новый за старый
}
void normalize(){                          // нормализация - удаление лишних 0
    while(n>0 && pd[n]==0) n--;
}
public:                                    // память не перераспределяется
    // метка открытой части
```

Метод **load** загружает копию массива коэффициентов в динамический массив в объекте. Но при этом он не освобождает память из под старого динамического массива. В конструкторах этого делать не нужно, а в остальных случаях – необходимо. Поэтому он является служебным (закрытым), поскольку его некорректный вызов может приводить к потере памяти. Методы **extend** и **normalize** изменяют размерность массива коэффициентов, перераспределяя соответствующим образом и память.

Рассмотрим еще один метод, интересный с точки зрения требований закрытости. Метод возвращает ссылку на выбранный коэффициент полинома, что позволяет работать с ним как по чтению, так и по записи. Хотя это «приоткрывает» доступ к внутренним данным объекта, но реальное использование этой ссылки «во вред объекту» и доступ через нее к другим коэффициентам требует большого искусства и не может быть произведено неосознанно (по ошибке). Поэтому такую операцию можно считать безопасной.

```
//-----101-01.cpp
// Класс степенного полинома
class poly{                                // закрытая часть (по умолчанию) – данные
    int n;                                // степень полинома
    double *pd;                            // динамический массив коэффициентов
public:
double &get(int k){                        // ссылка на коэффициент полинома
    static double foo=0;                  // вне пределов массива - ссылка
    if (k<0 || k>n) return foo;           // на "левую" статическую переменную
    return pd[k];}
```

Взаимодействие данных и алгоритма в ООП

Проблема «Что первично - курица или яйцо?» применительно к программированию звучит как «Что первично: алгоритм (процедура, функция) или обрабатываемые им данные». В традиционной технологии программирования взаимоотношение алгоритм (процедуры, функции) - данные имеют более-менее свободный характер, причем алгоритм является ведущим в этой связке: функция вызывает функцию, передавая данные друг другу по цепочке. Соответственно, технология структурного проектирования, прежде всего, уделяет внимание разработке алгоритма. Она может быть выражена одной фразой – **программирование (выполнение программы) «от функции к функции»**.

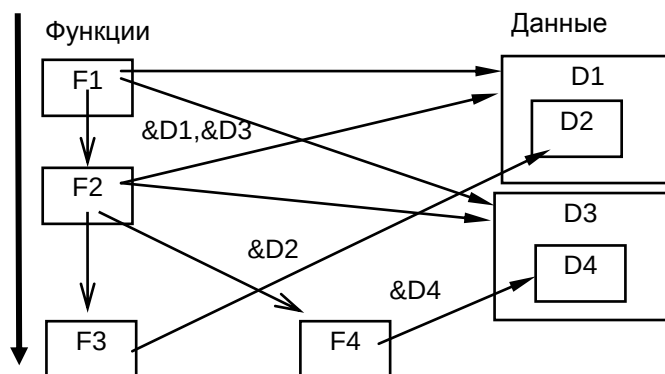


рис.101-3. Программирование «от функции к функции»

В технологии ООП взаимоотношения данных и алгоритма имеют более регулярный характер: во-первых, класс объединяет в себе данные и методы (функции). Во-вторых, схема взаимодействия функций и данных принципиально иная. Метод (функция), вызываемый для одного объекта, как правило, не вызывает другую функцию непосредственно. Для начала он должен иметь доступ к другому объекту (создать, получить указатель, использовать внутренний объект в текущем и т.д.), после чего он уже может вызвать для него один из известных методов. Следовательно, структура программы определяется взаимодействием объектов различных классов между собой, а процесс выполнения программы выражается фразой «**объект-метод-объект**».

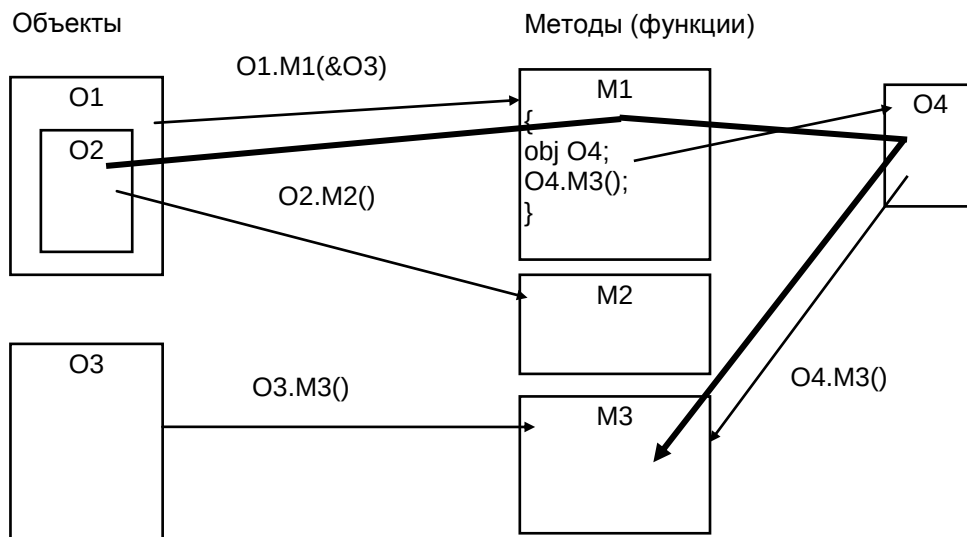


рис.101-4. Программирование в цепочке «объект-метод-объект»

Эпизодическое объектно-ориентированное программирование. Эпизодическое использование технологии ООП заключается в разработке отдельных, не связанных между собой классов и использовании их как необходимых программисту базовых типов данных, отсутствующих в языке. При этом общая структура программы остается традиционной «от функции к функции». Например, для работы с матрицами программист может определить класс матриц, переопределить для него стандартные арифметические операции и использовать переменные типа матрица в обычной программе.

Тотальное ООП и программирование «от класса к классу». Строгое следование технологии ООП предполагает, что любая функция в программе представляет собой метод для объекта некоторого класса. Это не означает, что нужно вводить в программу какие попало классы ради того, чтобы написать необходимые для работы функции. Наоборот, класс должен формироваться в программе естественным образом, как только в ней возникает необходимость описания новых физических предметов или абстрактных понятий (сущностей). С другой стороны, каждый новый шаг в разработке алгоритма также должен представлять собой разработку нового класса на основе уже существующих. В конце концов вся программа в таком виде представляет собой объект некоторого класса с единственным методом **run** (выполнить). «Тотальное» ООП (а именно оно является единственно возможным в полностью объектно-ориентированной среде типа Java или C#) основывается на представлении программы в виде системы классов, которые порождают взаимодействующие между собой объекты. Именно этот качественный переход в представлении программы (а не использование классов и объектов, как таковых) создает психологический барьер перед программистом, осваивающим технологию ООП.

Особенности модульного проектирования в технологии ООП

В 1.7 и 5.7 уже говорилось об особенностях разработки в Си больших (модульных) программ в виде набора независимо транслируемых модулей (файлов). Технология ООП и тут наводит порядок, объединяя понятия **модуль** и **класс**:

- каждый класс размещается в отдельном модуле, содержащем два файла: **заголовочный файл класса** с типом – **h** и файл, содержащий определения методов класса с типом – **cpp (файл тела класса)**;
- заголовочный файл не должен содержать конструкций языка, порождающих программный код, он должен целиком состоять из определений типов, объявлений переменных и функций и заголовка класса;
- в заголовке класса может присутствовать **объявление метода** – заголовок со списком типов параметров (прототип), ограниченный точкой с запятой. Это означает, что в заголовочнике упоминается только факт его наличия (с заданным именем и интерфейсом). Тогда в файле тела класса должно быть **определение метода**, содержащее его заголовок и тело. Заголовок повторяет объявление с одним маленьким отличием: имя метода дается в полной форме в виде **имя_класса::имя_метода**;
- второй вариант состоит в том, что в заголовке класса сразу же присутствует **определение метода** – заголовок и тело, заключенное в **{}**. Такой синтаксис более естественен, ибо позволяет «не разрывать на части» описание класса (например, такой способ единственно возможен в Java). К сожалению, по правилам трансляции в Си такой метод не является обычной функцией, а представляет собой подставляемый (**inline**) код. Транслятор вместо команды вызова функции включает в программный код копию тела метода, что «не есть

хорошо» хотя бы потому, что неэффективно. К тому же некоторые трансляторы имеют ограничения на синтаксис **inline**-методов;

- файл тела класса должен подключать свой заголовочный файл директивой **include**;
- для того, чтобы другой класс или **main** могли создавать объекты некоторого класса и применять к ним методы, необходимо подключать заголовочный файл директивой **include**;
- все имена заголовочных файлов и файлов тела класса должны быть включены в проект;

```
//-----Заголовочный файл A.h
class A{                               // Заголовок класса
    int a;                             // Данные класса
public:
    void add(A&); // Объявление (прототип) метода
    A mul(A&);   // Объявление (прототип) метода
    A& inc(){    // Встраиваемый (inline) метод
        a++;    // inline-код
        return *this;
    }
};
//-----Определение методов класса – файл A.cpp
#include "A.h" // Подключение собственного заголовочника
void A::add(A &T){ // Определение метода add -
    a+=T.a;       // заголовок с полным именем и тело
}
A A::mul(A &T){ // Определение метода mul –
    A tmp=*this;
    tmp.a*=T.a; // заголовок с полным именем и тело
    return tmp;
}

//----- Доступ из другого класса – файл B.h
#include "A.h" // Подключение заголовочника класса A
class B {
    void F(A& T){ // Использование объектов класса A
        A aa=T;  // в любом виде
    }
}

//----- Доступ из main – файл main.cpp
#include "A.h" // Подключение заголовочника класса A
void main(){
    A aa,bb;    // Создание объектов класса A
    aa.add(bb); // Работа с объектами класса A
}
```

Естественно, никто не запрещает «свалить все классы в одну кучу», не используя проекта. Для небольших программ это простительно, но не эстетично.

Лабораторный практикум

Разработать класс для требуемого типа данных: внутреннее представление данных, конструкторы, деструктор, методы ввода/вывода, изменения содержимого отдельных элементов. Необходимый материал для вар.14-18 см. в **9.1**.

1. Правильная дробь, представленная целой частью, числителем и знаменателем.
2. Целые числа, представленные в виде двух 32-разрядных переменных (**int**), хранящих 9 младших и старших десятичных цифр числа (т.е. части числа в диапазоне 0...999999999). Знак числа представлен отдельно.

3. Целое число, представленное в виде массива байтов. Каждый байт хранит 2 цифры числа (часть числа в диапазоне 0..99). Знак числа представлен отдельно.
4. Целое положительное число, представленное в виде массива его простых множителей (произведение которых дает это число).
5. Целое положительное число, представленное в виде массива остатков от деления на первые n простых чисел (представление числа в остаточных классах). Диапазон представления равен произведению первых n простых чисел.
6. Вектор на плоскости, представленный в полярной системе координат (длина, угол поворота).
7. Вещественное число в экспоненциальной форме: нормализованная дробная часть (в диапазоне 0.99...0.1) - double и целый показатель степени - int.
8. Матрица переменной размерности, представленная динамическим массивом указателей на строки матрицы (линейные динамические массивы).
9. Матрица переменной размерности, представленная динамическим массивом, в котором строки матрицы расположены последовательно друг за другом.
10. Разреженная матрица переменной размерности, ненулевые коэффициенты представлены динамическим массивом с элементами (x,y,v) координаты, значение.
11. Разреженная матрица переменной размерности, ненулевые коэффициенты представлены односвязным списком с элементами (x,y,v) координаты, значение.
12. Разреженная матрица переменной размерности, ненулевые коэффициенты представлены двусвязным циклическим списком с элементами (x,y,v) координаты, значение.
13. Множество, элементами которого являются целые числа. Операции объединения и пересечения множеств, добавления элемента, проверки на входжение, разности множеств.
14. Целые произвольной длины со знаком во внешней форме представления в виде строки цифр в прямом коде. Знак представлен отдельным элементом данных.
15. Целые произвольной длины со знаком во внешней форме представления в виде строки цифр в прямом коде. Знак представлен старшей цифрой (0 /1).
16. Целые произвольной длины со знаком во внешней форме представления в виде строки цифр в дополнительном коде.
17. Целые произвольной длины во внутреннем двоичном представлении (динамический массив байтов) в прямом коде. Знак представлен отдельным элементом данных.
18. Целые произвольной длины во внутреннем двоичном представлении (динамический массив байтов) в дополнительном коде.